

《Hello Armor：装甲板位姿解算》逐字讲稿

视觉组第四讲 · 承接 Lecture 3 《Hello Modern C++》· 90 分钟 · 配套 lecture4/yolo 工程

本讲稿对照《Lecture4_HelloArmor_装甲板位姿解算.pptx》(39 页)、《Lecture4_HelloArmor_教案.docx》与 lecture4/yolo 工程代码 (Task 01~05 全在 src/main.cpp) 编写，并完整覆盖《2025装甲板位姿解算》《Hello_Armor》两份原视频课的内容 (含欧拉角、万向锁、四元数)。

本届改用 YOLO 版程序：检测器是学生 lecture2 作业里那套 (tasks/ 与 tools/ 逐字复制，一行没改)。旧的手写传统 CV 版 lecture4/class/ 不再讲授，只作为代码保留——讲稿里不得把学生指向它。

使用说明

- 本讲稿中无标记的文字是教师可直接朗读的口播原文。
- > [...] 是舞台提示 (操作 / 板书 / 演示 / 停顿 / 应答)，不需要念出来。
- 每页标题后的 ⌚ 是该页建议口播时长。

时间轴总表

时间段	环节	幻灯片	时长
0:00-0:03	开场与课程地图	1-2	3 min
0:03-0:08	P0 复习导入	3-6	5 min
0:08-0:14	P1 为什么需要位姿	7-10	6 min
0:14-0:27	P2 PnP 原理与 API	11-14	13 min
0:27-0:51	P3 上机一 Task01~03 (含点序陷阱)	15-21	24 min
0:51-1:05	P4 rvec 揭秘与旋转表示法	22-27	14 min
1:05-1:21	P5 上机二 Task04~05 (含重投影自查)	28-32	16 min
1:21-1:27	P6 坐标系全景	33-35	6 min
1:27-1:30	P7 总结与作业	36-39	3 min
弹性	Q&A	39	弹性

合计：3 + 5 + 6 + 13 + 24 + 14 + 16 + 6 + 3 = 90 分钟。

提示 (时长口径)：第 2 页路线图上印刷的分段时长为 5/6/13/24/14/16/6/3，合计 87 分钟；加上开场 (第 1 页，2 分钟) 与路线图本身 (第 2 页，1 分钟)，共 90 分钟，与本表一致。

技术口径提示：Task 05 画面显示的欧拉角是**弧度** (当前代码未乘 57.3)；讲稿中出现的 57.3 仅作为"弧度换算成度数"的补充说明。

【幻灯片 1】封面 · 装甲板位姿解算 ⌚ 2 分钟

〔操作〕切到第 1 页封面，投影停住。全场第一句话从这里开始。

同学们好，欢迎回到视觉组的第四讲。我是这一讲的主讲人 _____，旁边这位是我们的助教 _____，等会儿上机的时候，大家卡在哪儿了，随时举手。

〔停顿〕等大家安静下来。

正式开始之前，先花半分钟，把这一讲的定位说清楚。

上一讲是第三讲《Hello Modern C++》。再往前那一讲我们干了什么？我们写了 Camera 类，写了检测器，最后让程序“看见”了装甲板——它能把那块装甲板认出来，把四个关键点画出来。

那这一讲《Hello Armor》，我们要在这个基础上往前走一大步：不光让程序“看见”装甲板，还要让它“算出”这块装甲板到底在哪儿、朝哪个方向。

〔板书〕右手边写下四个字：位姿解算。

大家看副标题——“从 2D 识别结果到 3D 位姿，cv::solvePnP 全流程实战”。说白了，就是把上节课的“2D 看见”，升级成今天的“3D 算出”。

那今天听完、做完，你手上会得到一个什么样的东西？是一个能跑起来的程序，能在画面上实时打印出三样东西：tvec、rvec，还有欧拉角 yaw、pitch、roll。

〔停顿〕这三个词现在听不懂，完全没关系。今天结束，你保证能自己写出来，也能自己讲明白。

还有件事提前打招呼：今天中间有两段时间是留给大家上机的，你们要自己改代码、编译、跑起来。现在先把电脑准备好。

好，那看今天的整体安排。

【幻灯片 2】课程地图 · Roadmap 🕒 1 分钟

〔操作〕翻到第 2 页，鼠标从 P0 依次指到 P7，走一遍。

这一页是今天的路线图，八个环节，我快速过一遍，大家心里有个节奏。

P0，复习导入，5 分钟，回一下你们 lecture2 作业里那套 YOLO 检测器。P1，为什么装甲板需要“位姿”而不只是“位置”，6 分钟。P2，PnP 原理和 cv::solvePnP 接口精讲，13 分钟。

P3，动手实验一，Task 01 到 03，解出 tvec、rvec，24 分钟——这是第一段上机，里头还塞了一页专门讲一个坑。

P4，rvec 揭秘，旋转向量、矩阵、欧拉角、四元数，14 分钟。P5，动手实验二，Task 04 到 05，把旋转变成看得懂的角度，16 分钟——这是第二段上机，末了给你一把自查的尺。P6，坐标系全景，6 分钟。P7，总结答疑和作业，3 分钟。

〔板书〕把 P3、P5 两个位置圈出来。

大家看，中间 P3 和 P5，就是给上机留的，加起来四十分钟。幻灯片上这八个数，5、6、13、24、14、16、6、3，加起来 87；再加上开场和这张路线图本身的 3 分钟，正好 90。

好，路线图就是这样。那我们从 P0 开始，回头看看：上节课结束的时候，我们手里到底有什么。

【幻灯片 3】P0 章节页：复习导入 · 上节课我们手里有了什么？ 🕒 0.5 分钟

〔操作〕翻到第 3 页，章节页多停一下，等学生抬头看投影。

好，我们正式进入今天的内容。这一讲是视觉组第四讲，主题是**装甲板位姿解算**。

在动手之前，我们先花几分钟，把上节课的东西捡回来。原因很简单——今天要写的代码，是踩在上节课的成果上的。

〔停顿〕

那我们开始，先回到上一讲。

【幻灯片 4】P0 · 上节课回顾 (Hello Modern C++) ⌚ 1.5 分钟

〔操作〕翻到第 4 页。

上上节课我们讲的是面向对象，上一讲是 Modern C++。这一页我不照着念，我直接考大家几个问题。

〔停顿〕两三秒。

第一个——讲面向对象那一节，我们自己动手写了哪两个类？

〔停顿〕等学生举手或者喊出来。

〔应答〕· 学生答「main 和 solvePnP」→「这两个都是函数，不是类。再想想——我们用 class 关键字写出来的是哪两个东西？」· 学生答「Camera 和 Detector」→「对，一个管相机，一个管识别。」· 学生答「相机类和一个检测器」→「没错，就是这两个方向。相机负责拿图，检测器负责在图里找装甲板。」· 冷场 → 老师自己接：「我提示一下——一个跟图像从哪来有关，一个跟图像里有什么有关。」

对，一个是 **Camera 类**，一个是**检测器**。

那我接着问第二个问题：Camera 类，对外提供了什么？

〔停顿〕

〔应答〕· 学生答「一张图片」→「对了一半。它给的是一帧图像，还带着一个时间戳。」· 学生答「返回图像和时间」→「非常准。图像加时间戳，一个都不能少。为什么要有时间戳？图像是滞后的，将来做运动预测，你得知道这张图是什么时候拍的。」· 冷场 → 老师自答：「它对外只给两样东西——一帧图像，加一个时间戳。取图的那些细节，全被它藏在里面了。」

这就叫**封装**——数据安全、隐藏实现、方便协作、防止耦合。这几个词你们听着像口号，但它就是我们写类的原因。

那检测器呢？你在 **lecture2 作业里做的就是这件事**——用神经网络识别装甲板，分出它的位置、颜色、数字。

〔停顿〕

这里我要强调一句，因为今天整堂课都建立在这上面：**今天用的检测器，就是你 lecture2 作业里跑通的那一套**。tasks/ 和 tools/ 两个目录我是原样复制过来的，一行没改。所以你今天打开工程，会觉得眼熟——那是你写过的代码。

好，上节课复习到这儿。

【幻灯片 5】P0 · 现在，程序手里已经有了什么？ ⌚ 1.5 分钟

〔操作〕翻到第 5 页，鼠标指着那个结构体关系图。

上节课复习完，我们来看一个特别关键的问题：**现在，程序手里已经有了什么？**

〔停顿〕

我们顺着数据捋一遍。

第一步，`detector.detect(img)`，把一帧图像喂进去。它返回什么？返回一个 `armors`，是一个 `Armor` 的列表——也就是说，图里可能有不止一块装甲板。

第二步，我们拿列表里的第一块，`armors.front()`。这个 `Armor` 结构体里，有一个成员叫 `points`——它就是 **4 个关键点**，顺序是左上、右上、右下、左下。

〔停顿〕让大家消化一下。

这里有个地方必须说清楚。你们翻一眼 `tasks/armor.hpp` 会发现，`Armor` 里还留着 `left` 和 `right` 两个成员，类型是 `Lightbar`——那是**传统灯条法**留下的历史包袱。走 YOLO 这条路，构造函数从来没给它们赋过值，你现在去取，拿到的是四个 `(0, 0)`。

记住这句话，待会儿 Task 02 就靠它保命：**本讲只用 `points`，不要碰 `left / right`。**

那 YOLO 是怎么给出这 4 个点的？**端到端，一步到位**——图片丢进去，类别和 4 个关键点一起出来。中间没有“先找灯条、再配对、再判断”这些步骤。这跟传统方法最大的区别就在这儿。

好，你数一数：一块装甲板，就是 **4 个 2D 像素点**。

这 4 个点，就是 lecture2 那一整套神经网络工作，最后交出来的东西。

换句话说——识别这一步，已经把「装甲板在哪张图的哪个位置」，彻底解决了。图里有几块板、每块板的四个角落在像素上的哪里，我们都拿到了。

〔板书〕`detect` → `Armor` 列表 → `points[0..3]` → 4 个 2D 点

这句话记一下，它是我们今天的起点。

【幻灯片 6】P0 · 只有这 4 个 2D 点，够瞄准吗？ ⌚ 1.5 分钟

〔操作〕翻到第 6 页。

那马上就有问题了。我们把手上所有的东西摊开来看：就这 4 个像素点。够不够让我们瞄准？

〔停顿〕

先看这两张图。

〔演示〕指投影上 3 号车和 4 号车的对比图。

这里是 3 号车，这里是 4 号车。谁离我们更远？

〔停顿〕等几秒。

〔应答〕·学生答「3 号车」→「对，一眼就看出来了。那我追问一句——你是怎么看出来的？」·学生答「4 号车」→「再仔细看看。3 号车的装甲板在画面上更小、位置更高，所以它才是更远的那个。人眼就是这么判断的。」·冷场→老师自己接：「答案是 3 号车更远，因为它看起来更小。」

人眼一眼就能判断距离。可问题来了——**程序呢？** 程序拿到的是那 4 个像素点的坐标，它能一眼看出远近吗？

〔停顿〕停一下，等学生反应。

答案是：**不能**。这 4 个点只说明了「它在画面上的哪个位置」，没说「它离我们有多远」。距离这个信息，在摄像机把三维世界投影成二维图像的那一瞬间，就被压扁、丢掉了。

再看第二组图，同一块装甲板，三张照片。

〔演示〕指向三张不同角度的装甲板。

这三块装甲板，有什么不同？

〔停顿〕

〔应答〕·学生答「角度不一样」→「对，就是角度。这就是朝向的差异。」·学生答「一块正着、一块歪着」→「说得很形象，就是这个意思。」·学生答「大小不一样」→「大小确实有差别，但我说的是同一块板、同一个尺寸，是角度不同才让它看起来不一样。再看一眼这三张图。」·冷场→「我换个问法——它是正对着我们，还是斜着的？」

角度不一样，朝向就不一样。而且你会发现，装甲板一歪，这 4 个像素点的位置也跟着变了。

所以，我们手里这 4 个像素点，要回答两个问题：**它离相机多远？它朝哪个方向偏？** 这两个问题，光靠二维坐标，答不上来。

〔操作〕准备翻到第 7 页。

这正是今天整堂课要解决的事。下面我们就来把它讲清楚。

【幻灯片 7】为什么需要位姿 ⌚ 0.5 分钟

〔操作〕翻到第 7 页，停在章节页，先别往下翻。

好，接下来就是今天正题的第一块——P1：为什么需要位姿。

〔操作〕鼠标指一下副标题“位置够不够？朝向怎么描述？”

看这个副标题，它一口气抛了两个问题：光知道位置，够不够用？就算要描述朝向，这个朝向又该怎么用数字说清楚？接下来这几页，就是专门回答这两个问题的。听完你就明白，我们为什么非要把“位姿”这件事折腾出来。

【幻灯片 8】位姿 = 位置 + 朝向 ⌚ 2 分钟

〔操作〕翻到第 8 页，鼠标点到“位姿”这个词上。

我们先看第一个概念。刚才一直说“位姿、位姿”，其实它是两个字拼起来的：一个是“位”，一个是“姿”。“位”就是位置，“姿”就是姿态，也就是朝向。

位置好理解——它就是装甲板中心，在相机坐标系下的一个三维坐标，写成 x 、 y 、 z 三个数。这三个数说白了，就是告诉相机：这个目标在你右边多少、下面多少、前面多远。

难的是第二个，朝向。装甲板“转了多少度、往哪边转”，这个怎么用数字说清楚？

〔停顿〕等两三秒，看有没有人回答。

〔应答〕·学生答“看它朝哪边歪”→“对，方向是这么回事。可‘歪’这个字程序听不懂，我们得把它变成数。”·学生答“转了多少度”→“很接近了。可往哪边转、绕哪根轴转，也得一并说清楚。”·学生答“朝向不就是它在哪吗”→“不对，这是两码事——位置管‘它在哪’，朝向管‘它脸朝哪边’，两个都得有，缺一个都算不出完整位姿。”·冷场→“我先卖个关子——它后面会用一个向量来表示，现在先把它的样子想明白就行。”

我借一个大家都熟的东西来讲——飞机。飞机在天上，除了往前飞，还有三个转动的方式，我们一个一个看。

〔演示〕双手张开当机翼，平举在胸前。

第一个，偏航，英文 yaw。它绕哪根轴？绕“上下”这根轴——你想象从飞机头顶往下穿一根竖着的杆子。绕这根轴转，改变的是前进的方向，也就是朝向的左右。

〔演示〕身体不动，双臂水平地整体向左、向右转。

第二个，俯仰，英文 pitch。它绕“左右”这根轴，也就是两个机翼连成的那根杆子。绕这根轴转，改变的是抬头、低头的程度。

〔演示〕双臂保持成一条水平直线，随上半身一起向前下方俯、再向后上方抬，模拟机头低下和抬起。

第三个，横滚，英文 roll。它绕“前后”这根轴，从机头指向机尾。绕这根轴转，改变的是侧向倾斜的程度——飞机往哪边歪。

〔演示〕双臂一上一下地翻转，像飞机往一边压坡度侧倾。

这三个词，yaw、pitch、roll，后面天天要用。

〔操作〕站直身体，用手比出三根轴的方向，放慢语速。

这里有一个特别关键、很多同学第一次都会漏掉的点：这三个转动不是随便转的，它们是按特定顺序定义的。顺序是——**先绕上下轴偏航，改变前进方向；再绕左右轴俯仰，改变俯仰程度；最后绕前后轴横滚，改变倾斜程度。**而且更重要的后半句是：**每一步都是在前面那一步转完之后的结果上再继续转**，不是三个轴各转各的、最后再加起来。顺序不能乱——你把顺序一换，同样这三个角度，转出来的姿态就不一样了。

〔板书〕顺序：先 yaw（绕上下轴）→再 pitch（绕左右轴）→后 roll（绕前后轴）；每一步在前一步的结果上转。

现在不用记公式，先把这个画面记住：**偏航管左右，俯仰管抬头低头，横滚管歪；而且三步有先后，一步踩一步。**

好，收一下——看出一块装甲板的三维信息，无非两样：**距离远近和朝向左右**。距离用位置这个三维向量描述，朝向就用刚才这三个角描述。可问题来了——**程序能自己解算吗？**想解算，得先给它一个参照的坐标系。翻页。

【幻灯片 9】先认清两个坐标系 ⌚ 2 分钟

〔操作〕翻到第 9 页。这一页今天特别重要，讲慢一点。

好，要把“位”和“姿”说清楚，得先有参照系。这一页认清两个坐标系——这两个你要是现在认混了，后面代码里的正负号全会跟着乱。

〔操作〕鼠标圈一下上半部分。

先看第一个，叫**装甲板局部坐标系**。注意，它是长在装甲板自己身上的，跟着装甲板一起动。原点在哪？就在装甲板的中心。三个轴怎么定？x 轴向右，y 轴向下，z 轴指向板子外面，也就是朝向镜头这一侧。你可以在脑子里把这块板子立起来：x 往右伸、y 往下伸、z 朝你这边戳过来。

〔操作〕鼠标移到两个数字上，放慢语速，这里让学生动笔。

然后是今天一定要用到的两个物理尺寸，大家拿笔记一下：灯条之间的宽度，`ARMOR_WIDTH`，等于 **0.135 米**；单根灯条的长度，`LIGHTBAR_LENGTH`，等于 **0.056 米**。**划重点——这两个数等下写代码的时候立刻就要用，就是上机实验里的 Task 01 要拿去搭那四个 3D 点。所以别听完就忘，先记下来。**它俩一横一竖，别搞反：0.135 是两根灯条之间的横向宽度，0.056 是单根灯条竖着的长度。

再看第二个坐标系，叫**相机坐标系**。它的原点在哪？在镜头的**光心**，也就是镜头正中间那个点。相机坐标系跟相机是绑死的：相机往哪平移，它就跟着平移；相机往哪转，它就跟着转。相机看到的一切，最后都要换算到这个坐标系里来描述。

〔停顿〕等学生把两个坐标系的定义和两个数字抄下来。

那我们把这两个坐标系摆在一起，目标是什么？一句话：算出“装甲板坐标系”相对于“相机坐标系”的**旋转 R** 和**平移 t**。这个 R 描述朝向，这个 t 描述位置，R 和 t 合起来，就是我们今天要的位姿。

好，两个坐标系立好了，目标也清楚了。那再追问一句——我们为什么非要姿态不可，光知道位置不行吗？翻页。

【幻灯片 10】工程上为什么必须要“姿态”，不只是“位置” ⌚ 1.5 分钟

〔操作〕翻到第 10 页。

到这儿可能有同学想：位置我知道了，朝向也大概懂了，可打比赛的时候，真非得要姿态吗？我光知道它在画面正中间，瞄着打，不行吗？

不行。这里有两个工程上的理由。

〔操作〕鼠标指第一条。

第一，相机和枪管往往不重合、也不共轴。你想，相机装在车顶，枪管在下面，这俩根本不是同一个点。那如果你只用画面正中心去瞄准，就会有一个系统性偏差——你永远偏那么一点点，而且这个偏差是固定的，不会自己消失。

〔操作〕鼠标指第二条。

第二，真实机器人是会自转、会移动的。用原课的话讲，“**实际车的运动状态千奇百怪**”——它在场上转来转去，姿态千变万化。这时候光知道它现在在哪没用，你得用它的姿态反推出它的旋转中心，才能预测它下一刻会转到哪、跑到哪。这就是我们后面做运动预测的基础。

所以记住这句话——**2D 检测解决“看见”，3D 位姿解决“打中”**。

〔停顿〕停两秒，让这句话落地。

看见，上节课已经做到了；打中，就要靠今天这一整套位姿解算。

那既然位姿这么重要，问题就来了：怎么从一张二维图像，反推出三维位姿？这就要请出今天的主角——PnP 原理。我们下一部分就来讲它。

【幻灯片 11】PnP 原理与 API · 从对应点到位姿 🕒 0.5 分钟

〔操作〕翻到第 11 页章节页，鼠标落在副标题"从对应点到位姿"上。

好，位姿这件事，我们用前面两段讲明白了：为什么会需要它、它由什么构成。那接下来这一段——P2——我们就来回答那个最核心的问题：这个位置和朝向，到底是怎么从一张图里算出来的？

这一段是今天理论上最"硬"的一段。但大家先别紧张，我们全程不推公式，只抓住一件事——**已知什么、求什么**。把这两头认清楚了，OpenCV 的接口你自然就会用了。走。

【幻灯片 12】Perspective-n-Points 问题 🕒 3.5 分钟

〔操作〕翻到第 12 页。〔板书〕黑板中间画一道竖线，左边写"已知"，右边写"求"。

这一页的名字叫 Perspective-n-Points，翻译过来就是"透视 n 点问题"，简称 PnP。名字听着唬人，我们把它拆成两半就懂了：Perspective 是"透视"，说的是相机成像的那个透视投影；n Points 是"n 个点"，说的是我们手里有 n 组对应的点。

我们就用刚才立的那个框架来拆——先看"已知什么"，再看"求什么"。

〔板书〕在"已知"那一侧写：n 组一一对应的点。

已知的第一样东西，是**一一对应的 n 组点**。什么叫一一对应？我们手里其实有两串点。

一串在**装甲板自己的局部坐标系**里，就是前面 P1 讲的那个"x 向右、y 向下、z 指向板外"的坐标系。它上面有 n 个三维点，这些点的坐标是我们自己定下来的，因为板子的真实尺寸我们知道——多大、点在哪，我们说了算。

另一串在**图像上**，是检测器已经给出来的 n 个二维像素点，也就是照片上那块装甲板、两根灯条的端点。

关键就在"对应"这两个字上：装甲板上第 1 个点，必须对应图像上第 1 个点；第 2 个对应第 2 个……一个萝卜一个坑，不能错位。这一点我们讲到接口的时候还会再强调一遍，因为它是新手最容易翻车的地方。

〔停顿〕两秒。

已知的第二样东西，是**相机的内参和畸变系数**。

内参是什么？你把它理解成"这个镜头把三维世界投影到照片上的方式"，主要就是**焦距和主点**。焦距管"东西在照片上被放大多少"，主点管"照片的中心落在哪个像素上"。换一个镜头、或者换一个分辨率去拍，同一块装甲板落在照片上的位置就不一样了，所以内参必须跟着"当前这个镜头、当前这个分辨率"走。

畸变系数呢，是因为真实镜头不是理想的小孔，画面边缘会有弯曲，畸变系数就是用来把这个弯曲修正回来的。

〔板书〕在"已知"那一侧补一行：相机内参（焦距、主点）+ 畸变系数。

好，已知说完了，我们看"求什么"。

〔板书〕在"求"那一侧写：旋转 R 和 平移 t 。

我们要求的，就是两样东西：一个**旋转 R** ，一个**平移 t** 。 R 管的是**朝向**——这块板子是怎么转的、往哪边偏； t 管的是**位置**——这块板子相对相机，摆在前方多远、偏左偏右、偏上偏下。 R 加 t ，合起来就是我们要的那个**姿势**。

那凭什么 n 个点就能把 R 和 t 反推出来呢？我打个比方，你马上就有感觉了。

假设你眼前有一个球，你知道这个球真实的直径有多大；然后你在照片上量一下它看起来有多大——你是不是立刻就能估计出它离你多远？它看起来越小，说明它离得越远。再比如，你知道一块砖的真实长度，又在照片上看见它斜着摆——那你多多少少能判断出它是朝哪边斜的。

这就是透视的基本直觉：**真实尺寸，加上它在照片上的样子，就能反推出"离多远"和"摆成什么角度"**。我们做的是一模一样的事情，只不过我们用的是四个点，要求也更严格一点——不光要"离多远"，还要"转了多少"。

好，这就是 PnP 问题。我把它拧成一句最短的话，你记一下：**已知 n 组一一对应的点，再加上相机内参和畸变系数，求物体相对相机的旋转 R 和平移 t** 。记住这个"已知什么、求什么"，下一页我们就看 OpenCV 怎么把它打包成一个函数。

【幻灯片 13】cv::solvePnP 函数签名精讲 ⌚ 5.5 分钟

〔操作〕翻到第 13 页，屏幕上出现 solvePnP 的签名。

好消息是——我们刚才讲的那一整套，OpenCV 已经全部封装好了，就一个函数：**cv::solvePnP**。

它的用法非常朴素：**你只管把数据"喂"进去，它把结果"吐"出来**。所以这一步我们不需要会数学，我们需要搞清楚的是——每个参数是什么、谁提供、长什么样。这一页我们一个参数一个参数过，慢一点，因为它就是你 Task 03 要写的东西。

〔操作〕用鼠标从头到尾划过六个必填参数，划一个讲一个。

```
bool cv::solvePnP(
    InputArray  objectPoints, // 输入：物体局部坐标系下的 n 个点
    InputArray  imagePoints,  // 输入：图像上对应的 n 个点
    InputArray  cameraMatrix, // 输入：相机内参矩阵（我们会提供）
    InputArray  distCoeffs,   // 输入：畸变系数（我们会提供）
    OutputArray rvec,         // 输出：旋转向量
    OutputArray tvec,         // 输出：平移向量
    bool useExtrinsicGuess = false,
    int  flags = SOLVEPNP_ITERATIVE
);
```

先看前两个。这是六个里最重要的两个，也是必须严格对应的两个。

第一个，**objectPoints**——**物体局部坐标系下的 n 个点**。谁提供？**你自己提供**，它就是 Task 01 里你要填的那个 **object_points**。它长什么样？它是一个点集，每个元素是 x 、 y 、 z 三个数。对装甲板来说， n 就是 4，也就是我们在装甲板坐标系里定义的四个点。

第二个，**imagePoints**——**图像上对应的 n 个点**。谁提供？**检测器提供，你来搬运**，它就是 Task 02 里你要填的那个 **img_points**，是从装甲板的端点里取出来的四个二维像素点。它每个元素是 x 、 y 两个数。

注意了——`objectPoints` 里的第 1 个点，和 `imagePoints` 里的第 1 个点，描述的必须是**同一个物理位置**。你 3D 点那边第一个填的是左上，2D 点这边第一个也必须是左上，顺序一点儿都不能岔。**这是整个 Task 03 里最容易出错的地方，没有之一。**这一页你要是只让我记一句话，那就记这句：**两个点集里，名次相同的点，必须是同一个位置。**

〔停顿〕两秒，让学生消化一下，顺便看看有没有人点头。

第三个，`cameraMatrix`——**相机内参矩阵**。谁提供？我们提供，代码里已经写好了，就在 `main.cpp` 里那个 `camera_matrix`，你不用自己标定。它长什么样？是一个 **3 乘 3 的矩阵**，里面装着焦距 `fx`、`fy` 和主点 `cx`、`cy` 这几个数。为什么要给你？因为 `solvePnP` 得知道“这个镜头是怎么成像的”，才能把照片上的像素点还原回空间里。

第四个，`distCoeffs`——**畸变系数**。谁提供？同样我们提供，代码里已经写好了，就是 `main.cpp` 里的 `distort_coeffs`。它长什么样？是一个 **1 乘 5 的向量**，也就是 5 个数，描述这个镜头在画面边缘的弯曲程度。

这两个“我们提供”的参数，你原样传进去就行，一个字都不用改。但你要明白它们为什么重要：**它们其实就等于“当前这个镜头、当前这个分辨率”的身份证，换了镜头或者换了分辨率，这两个就得重来。**

〔操作〕鼠标划到后两个必填参数。

再往下两个必填参数是**输出**——注意，是“输出”，也就是函数算完，往里填结果的。第五个 `rvec` 是**旋转向量**，第六个 `tvec` 是**平移向量**。你只要提前准备好两个变量，让函数往里写就行。它俩的细节，我们下一页专门拿出来讲透，现在先认识个脸熟。

〔操作〕鼠标划到最后两行带默认值的参数。

后面还有两个参数，`useExtrinsicGuess` 和 `flags`，都带着默认值。今天一句话带过：**用默认值，不用管。**前者默认是 `false`，意思是“我们不给它猜一个初始值”；后者默认是 `SOLVEPNP_ITERATIVE`，意思是“用它默认的迭代解法”。今天这两个都不用动。

〔板书〕按顺序写下六个参数：`objectPoints`, `imagePoints`, `cameraMatrix`, `distCoeffs`, `rvec`, `tvec`。

好，最后我把这个签名在黑板上抄一遍，同时我要求大家做一件事——**打开你的 `main.cpp`，在 Task 03 的注释里，把这个函数签名自己抄一遍**，六个参数就按这个顺序写：`objectPoints`, `imagePoints`, `cameraMatrix`, `distCoeffs`, `rvec`, `tvec`。为什么要抄？因为等你真到工程里调它的时候，参数顺序记错是家常便饭。**抄一遍，比看十遍都管用。**这也正好对应教案上给你们安排的那项活动，现在就动笔。

〔停顿〕留十几秒，等学生把签名落到自己的代码注释里。

【幻灯片 14】输出到底是什么？ ⌚ 3.5 分钟

〔操作〕翻到第 14 页，屏幕并排两张照片：装甲板坐标轴 + 相机坐标轴。

好，数据喂进去了，函数也调完了，它吐给我们的 `tvec` 和 `rvec` 到底是什么？这一页，我们把这两个输出讲透——这两个词，你接下来会看一整个下午，必须弄清楚。

先说 `tvec`。它是一个 **3 乘 1 的向量**，代表**装甲板坐标系的原点，在相机坐标系下的位置**。这句话拆成两个关键点。

第一，“装甲板坐标系的原点”——还记得吧，我们把原点放在了板子的正中心。

第二，"在相机坐标系下"——也就是说，它回答的是：**装甲板中心，在相机前面多远、偏左还是偏右、偏上还是偏下**。这三个数，就是这个意思。

所以你也可以把 **tvec** 直观地理解成**平移向量**：把装甲板中心从镜头光心那里，平移到它现在这个位置，要往 x、y、z 各走多少。它是位置信息，跟旋转没关系。

〔停顿〕两秒。

再说 **rvec**。

这里我要**先卖个关子**。我现在可以告诉你的是：**rvec** 也是一个 **3 乘 1 的向量**，三个数，它代表的是**旋转**——也就是这块装甲板的朝向是怎么转的。

但具体这三个数各自是什么意思、怎么就从三个数读出一个旋转，我这一页**故意不展开**。为什么？因为原课里就是这么处理的，我也是这么学的——**先写代码，留个悬念，后面再来解答**。你先把 **rvec** 当成一个"三个数的黑盒子"，先会用、先让它转起来，等你亲手把装甲板转一转、亲眼看它三个数怎么变，那时候再讲定义，你一下就通了。

〔停顿〕两秒，把悬念留住，别急着往下说。

这个悬念，就是 P4 我们要专门解开的东西——到那一页，我们会把 **rvec** 拆开来看：它的方向是什么、大小是什么，它又怎么变成我们看得懂的 yaw、pitch、roll。所以现在，你只要记住一句话：**tvec 管位置，rvec 管朝向，两个都是 3 乘 1 的向量，一个是结果，一个是伏笔**。

好，PnP 的原理和接口，到这里就讲完了。接下来——该你们动手了。我们进入 P3 上机实验一，把 Task 01、Task 02、Task 03 一路写出来，先让 **solvePnP** 把 **tvec** 和 **rvec** 解出来。注意，这一步画面上暂时还是 0.00——想让这两个数真正显示出来，那是 P5 的事，我们先把这一步跑通。翻页。

【幻灯片 15】P3 章节页 · 动手实验一：Task 01~03 ⌚ 0.5 分钟

〔操作〕翻到第 15 页，停在章节页，把投影切到「IDE + 终端」分屏。

好，抬头看一下这一页。从这儿开始，进入今天第一段真正动手的时间。接下来二十四分钟，讲台上的声音会变少，键盘的声音会变多——三个任务，Task 01、Task 02、Task 03，要你们自己敲出来。

做完这三步，你的程序就能从一幅图像里的四个像素点，反推出装甲板相对相机的三维位姿。位姿的两个结果，一个叫 **tvec**，一个叫 **rvec**，都会在程序里算出来——不过现在还不会显示到画面上，那是后面的事。先看下一页，把环境和规矩讲清楚，再动手。

〔操作〕切到第 16 页。

【幻灯片 16】实验环境 & 点位约定 ⌚ 4 分钟

先说清楚今天要动哪几个文件，别自己乱翻。

打开工程目录 **lecture4/yolo**。今天你只有一句话要记：**所有要填空的地方，都在 **src/main.cpp** 这一个文件里**。Task 01 到 Task 05，全在这一个文件里，你从头往下顺，不会跑偏。

〔停顿〕

tasks/ 和 **tools/** 两个目录，我不让你们动——而且你们应该看着眼熟：**那是我从你们 lecture2 作业里原样复制过来的，一行没改**。所以今天你不是在学一个新框架，你是在自己写过的代码上，再加一步。

第二件事，运行目录。 这个工程里，配置文件、模型、视频、日志，四样东西都是按相对路径找的，所以你必须站在 `lecture4/yolo/` 目录下运行程序。跑到别的地方去，它会报错——等一下演示的时候我们现场踩一遍，先埋个伏笔。

第三件事，一个约定。 工程里有 `answer/main.cpp`，那是我的参考实现，里面 Task 01 到 05 全是填好的。我把它默认设成不编译了，但文件就在那儿。

〔停顿〕扫视一圈。

这二十四分钟，**不要打开 `answer/main.cpp`**。卡住了举手，我过来看。等你自己的 `./build/main` 跑起来了，我在投影上跑参考版给你们对照——它的画面上会比你们多一行数，那一行待会儿有用。

〔操作〕鼠标圈住幻灯片上点序那一行。

接下来是最关键的一条，也是今天最容易翻车的一条：**点序**。

我们从头到尾只用一个顺序——**左上、右上、右下、左下**，自左上起顺时针绕一圈。物理上它们就是左灯条顶端、右灯条顶端、右灯条底端、左灯条底端。

〔停顿〕

但注意后半句：**程序里已经没有「灯条」这个对象了**。YOLO 直接给你 4 个关键点，没有中间那一步。所以你在脑子里按实物记顺序可以，在代码里不要去找灯条。

Task 01 你在三维里排的四个点，和 Task 02 你在图像里排的四个点，必须一个对一个、完全对齐：第 1 个 3D 点对第 1 个 2D 点，第 2 个对第 2 个，顺次往下。这个约定没有商量的余地。

为什么？因为 `solvePnP` 这个函数，它自己不知道哪个点在左、哪个在右。它只知道「你给我的第 i 个 3D 点，对应你给我的第 i 个 2D 点」。顺序错了，它照样给你一个解——但那个解是错的，而且它不报错。

〔演示〕举起装甲板实物，用手指依次点过左上、右上、右下、左下。

我现在建议你们马上做一件事：拿起实物，或者拿屏幕上的照片，用手指按左上、右上、右下、左下划两遍，把这个顺序刻进肌肉记忆里。等一下敲代码的时候，就不容易错了。

好，记住这个顺序，我们看第 17 页，Task 01。

【幻灯片 17】Task 01 · 填写 `object_points` ⌚ 5 分钟

〔操作〕翻到第 17 页，切到 IDE，打开 `src/main.cpp`，光标停到文件开头的 Task 01 注释处。

Task 01 在 `main.cpp` 的文件顶部，大概二十几行的位置。

先说一个**很多人会卡住的细节**：这一段模板，现在是**整段被双斜杠注释掉的**，你在编辑器里看到的是一片绿字。所以第一步不是填数字，是**先解注释**——把它前面的 `//` 去掉，让它变成真正生效的代码。这个提醒很重要，不然后面 `solvePnP` 会报「找不到 `object_points` 这个名字」，你还以为是别的地方错了。

〔操作〕指幻灯片上的模板代码块。

它现在在文件里是这样——整段都被双斜杠注释着：

```
// static const std::vector<cv::Point3f> object_points {  
//     {           ,           , 0 }, // 点 1
```

```
//      {      ,      , 0 }, // 点 2
//      {      ,      , 0 }, // 点 3
//      {      ,      , 0 }  // 点 4
// };
```

你要填的是每个点的前两个空——第一个是 x ，第二个是 y 。

这个坐标系叫**装甲板局部坐标系**：原点在板的正中心， x 轴向右为正， y 轴向下为正。因为四个角点都躺在同一张平板面上，所以它们的 z 分量全是 **0**，这个 0 已经帮你填好了。

〔操作〕鼠标框住幻灯片上 `ARMOR_WIDTH / 2` 那一句。

代码注释里明确要求：用「 $\pm \text{ARMOR_WIDTH} / 2$ 」这样的写法来填，不要写死小数。为什么？因为装甲板的尺寸会改，你写成 0.0675 这种硬编码，尺寸一改你全得重来；写成表达式，尺寸改了它自动跟着变。宽度方向用 `ARMOR_WIDTH / 2`，高度方向用 `LIGHTBAR_LENGTH / 2`——一个是半宽，一个是半高。

现在我们一起推，我不给答案。

〔板书〕黑板上画一个矩形，标出中心，画上向右的 x 轴、向下的 y 轴。

〔停顿〕等三秒。

第 1 个点，左上角。它相对板中心，是偏左还是偏右？偏左的话， x 应该是正的还是负的？

〔停顿〕

它在上方。 y 轴是朝下的——越往下 y 越大——那最上面这个点， y 是正的还是负的？

〔停顿〕

〔应答〕·学生推出正确的符号 → 「对，这是你自己推出来的。思路就对了：先看这个点落在中心的哪一侧，再对照轴的方向。剩下三个点，照这个来。」·学生把符号说反了 → 「先别急着背结论。把 x 轴、 y 轴画在纸上，看这个点落在轴的哪一侧。 y 轴朝下、越往下越大——那最上面这个点的 y ，应该更大还是更小？」·冷场 → 「我给思路不给答案：先别管正负，把绝对值填进去，四个点都是半宽和半高；正负号，就看你这个点在中心的哪一侧。剩下的你们自己推。」

好，剩下的三个点，照同样的道理自己推。我不念答案。

再提几个我见过的常见错误，你们避开：

第一，**正负号搞反**——最常见。第二，**把 0.135 当成半宽**——0.135 是灯条中心间距、是两个灯条中心的距离，半宽是它的一半，别再除第二次。第三，**忘了解注释**，就是你前面遇到的那个。第四，**顺序写错**——四个点的注释都给你标了号，别填岔行。

〔操作〕切到 `tasks/armor.hpp`，指一下 `Armor` 结构体里的 `center`。

还有一句提醒：**`Armor` 结构体里那个 `center`，不是装甲板四边形的角点**。它是那 4 个关键点的平均值，也就是这个四边形的形心——一个算出来的虚拟点，不是四个角里的任何一个。PnP 要的是能和物理 3D 点一一对应的那四个真实端点，别拿 `center` 来凑数。

〔停顿〕

好，四个点填完了，我们歇一分钟，讲一个坑。翻到第 18 页。

【幻灯片 18】点序陷阱：注释是错的 ⌚ 4 分钟

〔操作〕翻到第 18 页，投影切回 IDE，打开 `tasks/armor.hpp`，光标停到 `points` 那一行。

这一页是今天我最想让你带走的东西，它不是知识点，是一种判断力。

看这一行——`tasks/armor.hpp` 里，`Armor::points` 后面跟着一条注释，写的是「关键点的图像坐标，顺序为左上、左下、右下、右上」。

〔停顿〕让大家看一眼。

这条注释是错的。

实测下来的真实顺序是**左上、右上、右下、左下**——自左上顺时针。注释写的那个是逆时针，正好反了。

〔停顿〕等这个信息落地。

你可能会想：那我按注释写不就行了？不行。**这就是今天最危险的一个坑**——顺序写反了，你的程序编译通过、画面正常、`tvec` 也照样给你一组数。它不报错，不崩溃，什么都不提示。你只有等到后面发现位姿莫名其妙不对，才会回头怀疑这里。

那我怎么知道的？两种办法，现场都能做。

第一种，三秒钟就能看出来。 程序里那行 `tools::draw_points(img, armor.points)` 已经在画那四个点了。你在这四个点旁边按下标标上 0、1、2、3，跑一遍，肉眼一看就明白谁是左上、谁是左下。

〔操作〕切到运行中的程序，指一下画面上标注的四个点。

第二种更硬，也更值得学。 把 `object_points` 换成注释里写的那个顺序，重新编译一次——然后看 `reproj err` 这个数。顺序对的时候它是个位数，写反了它会跳到几十。

〔停顿〕

`reproj err` 是什么，我在 P5 会专门讲一页。这里你先记住结论：**它是唯一能一眼判死点序对错的工具。**

最后回到这条注释本身。它为什么会错？我猜是抄的时候没改——**因为 `armor.hpp` 里这条注释出现了两次**，文件头的信息块里有一次，成员那一行又有一次。错误的东西被复制了一遍，看起来就更像真的了。

〔板书〕注释是人写的，会过期；代码和数据不会撒谎。冲突的时候，信实测。

这句话记一下。以后你写代码、读别人的代码，都用得上。

好，翻回第 19 页，我们继续 Task 02。

【幻灯片 19】Task 02 · 填写 `img_points` ⌚ 4 分钟

〔操作〕翻到第 19 页，切回 IDE，光标停到主循环里 Task 02 的注释处。

Task 02 还在 `main.cpp`，但位置换了——**Task 01 在文件开头，Task 02 到 Task 05 都在主循环里面，就是装甲板检测到之后的那一段。** 注意这个位置关系。

Task 02 要做的是：把检测器已经给出的四个关键点，按同样的左上、右上、右下、左下顺序，装进 `img_points` 这个向量里。

〔操作〕鼠标指到幻灯片提示里 `armor.points` 那一行。

好消息是：这四个点已经躺在 `armor.points` 里了，而且顺序就是我们要的那个。你要做的是把它们取出来，一行一个，写进 `img_points`。

〔停顿〕

那怎么取？`armor.points` 是个 `std::vector<cv::Point2f>`，用 `.at()` 或者 `[]` 都行——`.at(0)`、`.at(1)`、`.at(2)`、`.at(3)`。注意下标从 0 开始，这是 C++ 的老规矩，跟我们嘴上说的「1 号点、2 号点」差一位。

我建议你写的时候，四个 `.at()` 后面都跟一句注释，把「左上」「右上」「右下」「左下」标出来。这样你回头检查的时候，一眼就能对上。

〔停顿〕给大家两秒扫一眼代码。

现在说一件更重要的事，也是今天最容易让你白干一小时的事。

〔操作〕切到 `tasks/armor.hpp`，指一下 `left / right` 两个成员。

你们刚才翻 `armor.hpp` 的时候，肯定看见了——`Armor` 里还有 `left` 和 `right` 两个成员，类型是 `Lightbar`。你会很自然地想：那四个点是不是应该从 `armor.left.top`、`armor.right.top` 这些地方取？

不要。这两个成员在本讲走的这条路上，从来没被赋过值。

〔停顿〕等这句话落地。

我再讲一遍，因为这个坑太隐蔽了：`Armor` 的构造函数有五个重载。传统灯条法用的是其中一个，那个确实会填 `left` 和 `right`。但 YOLO 这条路用的是另一个——它只填 `confidence`、`box` 和 `points`，`left / right` 走的是默认构造，里面四个坐标全是 `(0, 0)`。

所以你要是写 `armor.left.top`，你会拿到一个 `(0, 0)`。四个点全是 `(0, 0)`，重合在一起。

〔停顿〕

会发生什么？`solvePnP` 拿到四个重合的点，大概率直接抛异常——程序闪退；也可能不抛，给你一组大得离谱的数。关键是：你在画面上看不出来。因为画那四个点的代码在 Task 02 前面，用的是 `armor.points`，照样画得好好的。你看着屏幕上四个点标得端端正正，根本想不到问题出在 `img_points` 上。

〔应答〕·学生问「那我怎么知道我用错了？」→「问得好。症状是：画面上的点画得好好的，但程序一跑就崩，或者数值乱跳。记住这个组合——点是对的，数不对，那就是取错地方了。」·学生问「为什么不把 `left/right` 删掉？」→「因为传统灯条法那条路还要用。这是历史包袱——代码里留着的东西，不等于现在能用的东西。」·冷场 →「我直接给结论：只用 `points`，别碰 `left / right`。这两个成员今天当它不存在。」

好，再确认一遍：`img_points` 里填的是 `armor.points` 的四个元素，顺序左上、右上、右下、左下。

最后再提醒一句刚才那个问题：如果你把其中两个点对调了，会发生什么？

〔停顿〕

〔应答〕·学生答「会报错」→「不会报错，这才是最坑的地方。`solvePnP` 照样给你一个输出，程序一点都不崩。但它算出来的位姿是错的。」·学生答「点会乱」→「思路对。它不崩，但解出来的位姿是

错的——这是最难查的一类错，因为它安安静静。」·冷场 → 「我告诉你们答案：程序不报错，但位姿是错的。所以点序这条，从头到尾都得盯死。」

好，Task 02 完成，看第 20 页，把这两组点交给 solvePnP。

【幻灯片 20】Task 03 · 调用 cv::solvePnP ⌚ 3 分钟

〔操作〕翻到第 20 页，切回 IDE，光标停到 Task 03 的注释处。

Task 03，还在 `main.cpp` 主循环里，就在 Task 02 下面。前面两个任务的产出，在这里汇合。

```
// #### Task 03 #####
cv::Mat rvec, tvec;
// 所有要传入的值都已经具备了。现在调用 solvePnP 解算装甲板位姿，
// rvec 和 tvec 用于存储 solvePnP 输出的结果。
// 你需要在下面填写 输入给 solvePnP 的参数：
//
// cv::solvePnP(, , , , rvec, tvec);
//
// #####
```

看清楚这个模板：`rvec` 和 `tvec` 代码里已经帮你提前声明好了，`cv::Mat rvec, tvec;` 这一行就是。它们是输出参数——你调用 `solvePnP`，函数会把解出来的结果填进这两个变量里。你要做的是把那四个空填对。

哪四个空？按顺序：第一个是 `object_points`，你 Task 01 填的 3D 点；第二个是 `img_points`，你 Task 02 填的 2D 点；第三个是 `camera_matrix`，第四个是 `distort_coeffs`。

后两个你不用操心。相机内参 `camera_matrix` 和畸变系数 `distort_coeffs`，已经在 `main.cpp` 文件最顶上给你写好了，直接传进去就行。那两行上面还有 `clang-format off` 保护着，就是怕被格式化工具打乱，你也不用动它。

〔操作〕鼠标指到签名里的参数顺序。

唯一要提醒的就是顺序。3D 点、2D 点、内参、畸变——前面这四个是输入，`rvec`、`tvec` 放最后，是输出。顺序不能错，错一个位置，要么编译不过，要么解出来是垃圾。

写完 Task 03，就可以编译运行了。看第 21 页，我现场跑一遍。

【幻灯片 21】现场演示 & Debug 小贴士 ⌚ 3.5 分钟

〔操作〕翻到第 21 页，切到终端，投影放大，边操作边讲。

好，看我的终端。进入工程目录，一共三步。

```
cd lecture4/yolo
cmake -B build
cmake --build build
./build/main
```

第一步进目录、配置，第二步构建，第三步运行。注意最后一步——运行的时候，你要站在 `lecture4/yolo` 这个目录下，敲 `./build/main`，也就是从项目根目录去运行 `build` 里的可执行文件。

〔演示〕切到一个错误目录，`cd` 进 `build/` 里，敲 `./main`，让报错刷出来。

现在我故意踩个坑给你们看。假设你 `cd` 进了 `build` 目录，然后直接敲 `./main`——回车。看这个报错：

```
terminate called after throwing an instance of 'YAML::BadFile'
  what(): bad file: configs/yolo.yaml
Aborted (core dumped)
```

先注意一件事：报错里说的是 `configs/yolo.yaml`，不是视频。为什么？因为程序一上来先构造检测器，检测器要先读配置文件；视频是后面才打开的。报错的顺序，反映的是构造的顺序。这本身就是个有用的调试直觉。

原因也很简单：`configs/yolo.yaml` 在项目的根目录，也就是 `lecture4/yolo` 下面，它不在 `build` 里面。你人在 `build` 目录里跑程序，程序按当前目录去找配置，当然找不着。

〔演示〕`cd ..` 回到 `lecture4/yolo`，再敲 `./build/main`，窗口正常弹出。

解决的办法就一句话：`cd` 回 `lecture4/yolo`，再敲 `./build/main`。看，窗口出来了。

顺便记住这个工程里有四样东西都是按相对路径找的：配置文件、模型、视频、日志目录。所以「站在正确的目录下运行」这条，不是可有可无的习惯。

〔操作〕指屏幕，画面里装甲板轮廓被画出来。

跑通之后，画面上会把检测到的装甲板轮廓画出来——就是那四个关键点连成的框。这时候 `tvec` 和 `rvec` 其实已经在程序里算出来了，只是还没显示到画面上，那是下一环节 Task 04、Task 05 的事。

〔停顿〕这里要停一下，因为接下来这句话很容易被误解。

但是——这个框画出来了，跟你 Task 01 到 03 填得对不对，一点关系都没有。

〔停顿〕等大家反应过来。

画框那行代码是 `tools::draw_points(img, armor.points)`，它在 Task 02 的代码块前面。也就是说，你 Task 01 的模板继续注释着、Task 02 和 Task 03 一个字都不填，程序照样编译、照样画框。

所以别拿「框出来了」当作做对了的证据。真正的验收在后面：等 Task 04 把数字打出来，你要看的是——`tvec` 的 `z` 跟着画面里装甲板的远近连续变、不为零、不乱跳。那才算走通。

好，原理和坑都讲完了，现在开始动手。哪一步卡住就举手，我下来巡场。

【巡场话术库】学生敲代码时，老师该说什么

〔操作〕老师走下讲台，逐个工位巡视。以下按卡点分类。

卡点零：`cmake -B build` 就过不去。

「先分清是哪个阶段错的。如果是 CMake Error，那是配置期的问题，跟你 `main.cpp` 写什么完全无关，别去改代码。」

「缺依赖的话，报错里会点名是哪个包找不到。千万别把 `find_package` 那行注释掉——那只是把错误藏起来，后面链接一样过不去。」

「如果是 `OpenVINO_DIR` 相关的，改顶层 `CMakeLists.txt` 里 `set(OpenVINO_DIR ...)` 那一行，改成你机器上的实际路径。」

「还有一种：编译过了，一运行报 `error while loading shared libraries: libopenvino.so.*`。那是运行期找不到库，`source /opt/intel/openvino_2024.6.0/setupvars.sh` 就好。」

卡点一：Task 01 找不到要填的地方（模板被注释了）。

「你往上翻——文件开头二十几行，有一段从 `static const std::vector` 开始、整段都是双斜杠的绿字，那就是它。先把它前面的 `//` 去掉，解注释，再填四个坐标。」

「找不到就用编辑器的搜索，搜 `object_points` 这个词，一下就跳过去了。」

「别只删注释符号就不管了——四个点的坐标还是空的，得你自己推。」

卡点二：Task 02 取点取错地方了（今天最隐蔽的坑）。

「你是不是写了 `armor.left.top` 或者 `armor.right.top`？那两个成员今天从来没被赋过值，你拿到的是四个 `(0, 0)`。」

「YOLO 走的构造函数只填 `confidence`、`box` 和 `points`，`left / right` 是传统灯条法的遗留，走这条路根本没碰过。改用 `armor.points.at(0)` 到 `.at(3)`。」

「症状很好认：画面上的点画得好好的，但程序一跑就崩、或者数值乱跳。点是对的、数不对，那就是取错地方了。」

卡点三：编译不过。

「报错先看第一行，别看最后一行。C++ 的报错经常一环套一环，最上面那个往往才是根因。」

「是不是 `object_points` 还在注释里？你前面忘了解注释，后面一用这个名字就报找不到。」

「如果报的是 `cannot convert 'cv::Scalar' to 'double'`——那是 `draw_text` 的参数顺序写反了。这个工程的顺序是 `(img, text, point, color, font_scale, thickness)`，颜色在前、字号在后，跟你们可能见过的别的写法不一样。这个我们待会儿 Task 04 会专门讲。」

卡点四：画面里没有装甲板轮廓。

「先确认视频在不在正常播放。注意：视频是循环播的，画面一直在动是正常的，不代表检测没问题——所以别拿『画面在动』当判断依据。」

「如果确实没框，去调 `configs/yolo.yaml` 里的 `min_confidence`，默认 0.8，想先看到框可以调到 0.7 试试。但别往 0.7 以下调——代码里还有一道写死的 0.7 更早一步把低分滤掉了，要再往下就得改头文件重编译。」

「别去动 `threshold` 和 `use_traditional` 这两个参数——它们是 lecture2 留下的历史字段，代码里已经不用了。改了不会有任何变化。」

「顺带说一个通用教训：配置文件里的参数，不一定是最终生效的那个。跟今天那条错误注释是同一件事——别只看声明，要顺着代码找到真正被用的那一行。」

【集合收尾话术】

〔操作〕看表，剩最后半分钟，拍两下手，把学生注意力拉回投影，同时切到第 22 页。

好，手先停一下，抬头看投影。不管你的框画出来没有，先把进度停在这儿。跑通的同学，恭喜你——你已经打通了「像素点 → 相机坐标系位姿」这最关键的一环；没跑通的也别慌，代码留着，我们后面还要接着用，先跟着往下听。

我来收一句。刚才你们做的事，本质上就三步：**在装甲板自己的坐标系里，定义清楚四个 3D 点在哪；在图像里，按同样的顺序找到对应的四个 2D 点；然后把这两组点、连同相机内参，一起交给 solvePnP，让它反推出装甲板相对相机的旋转和平移。**

平移就是 **tvec**，三个数；旋转就是 **rvec**，也是三个数。它们现在已经在你的程序里解出来了，只是还没打到屏幕上。

但是——你有没有想过一个问题？**rvec** 这三个数，它到底在说什么？是方向？是角度？为什么偏偏是三个，不是四个、也不是九个？

〔操作〕把投影从 IDE 切回 PPT。

这就是我们下一段要揭开的谜底。**rvec** 揭秘。

【幻灯片 22】P4 章节页 · rvec 揭秘 ⌚ 0.5 分钟

〔操作〕翻到第 22 页，停在 P4 章节页。

〔操作〕切到教师机终端/画面窗口，投影上显示出 **rvec** 那三个数值。

好，先把手上的代码放一放，抬头看投影。刚才上机的时候，你们的程序其实已经把两个结果算出来了——一个叫 **tvec**，一个叫 **rvec**。只不过那会儿还没打到画面上，你们屏幕上那两行数字现在还是 0.00。现在我在投影上把它们打出来，我们边看边说。**tvec** 那三个数很好懂，它是位置，装甲板一动它就跟着变，这个我们前面讲过。可旁边还有三个数，**rvec**，**x**、**y**、**z**，它也随着装甲板一起变。

那这一节，P4，我们就专门解决一件事：**rvec 这三个数，到底是什么？**

注意，这一节是本讲的理论核心——欧拉角、万向锁、四元数全在这里，大家打起精神。

【幻灯片 23】转一转，看一看 ⌚ 3 分钟

〔操作〕翻到第 23 页，标题是“转一转，看一看”。投影切到教师机正在运行的程序窗口。

这一页我不讲，我们做个小观察。先说怎么做。

〔演示〕指着投影上正在运行的程序窗口——注意，这是教师机（用 **-DBUILD_ANSWER=ON** 编出来的参考版）在跑，不是你们自己现在这份程序。

看这个窗口。**左上角有三行数**——第一行 **tvec**，第二行 **rvec**，第三行是欧拉角，那是后面才讲的。我们今天只盯**第二行 rvec**。

〔停顿〕让大家找到那三行数。

画面上那只手拿着的，就是一块装甲板。程序默认在循环播放 **assets/video.avi**——那里面就是一个人手持装甲板在动，25 秒一轮，所以等它转回来不用很久。

〔停顿〕

好，现在我们一起盯着 **rvec** 那三个数。注意两件事：

第一，**这些数就在画面左上角，原地刷新，不滚动**。我不切终端，因为程序根本不往终端打印——你们看投影就行。

第二，**数值都很小**，零点几这种小数。为什么这么小？因为**单位是弧度**，不是度。这个待会儿要专门讲。

现在看那只手——他正把装甲板慢慢地、**只绕着一个轴左右扭**。我们盯着 **rvec** 的 x 、 y 、 z 三个数，看哪个在动、哪个基本不动。

〔停顿〕让视频转一轮，指一下画面里数值的变化。

如果你想停在某一帧上仔细看，**按一下空格键**，画面就暂停了；再按一下继续。

〔停顿〕转一到两轮，让大家看够。

好，停。现在我不给答案，先请你们猜。还记得第 14 页我卖的关子吧？当时我说，**rvec** 也是一个三乘一的向量，代表旋转，但**具体是什么，先卖个关子**——现在到兑现的时候了。

第一个问题——

「只绕着一个轴转的时候，**rvec** 的三个分量里，是哪一个在变？」

〔停顿〕等五六秒，看看有没有人举手。

〔应答〕·学生答“是中间那个 y 在变” → “对，你观察得很准。我再追一句——为什么偏偏是它？先别急着解释，记住你这个直觉，下一页我帮你验证。”·学生答“三个都在变” → “这个答案很正常，我第一遍看也觉得是乱的。你再看一轮——有没有哪个数字其实动得很小、几乎没动？”·学生答“我不知道” / 冷场 → “没关系，我们一起再看一轮，这次你只盯着中间那个数看，它动没动？”

第二个问题，还是靠眼睛看——

「转得越多，这个数字的变化有什么规律？它是一下子跳过去，还是慢慢变大？转到多少的时候它最大？」

〔演示〕按空格暂停在几个不同姿态上，对比数值；再继续播放。

〔停顿〕让大家观察数值从小到大、再从大到小、甚至变号的过程。

〔应答〕·学生答“它是慢慢连续变的，不是跳的” → “非常好，**连续**这两个字你抓住了，这是关键。”·学生答“转到某个角度之后它又变小了” → “哦，这个观察很关键——说明它不是一个一直往上爬的数。那它会不会一路变到负的？”·学生答“它会变号” → “对！变号这件事特别有意思。你想想，什么时候一个数会从正翻到负？”·学生答“它一直变大，不会变小” → “好，那你别急，等它转到下一个姿态——盯住这个数，看它会不会突然往回跳？”·冷场 → “我给个提示：注意这个数到某个角度以后符号会翻过去。你们看，它从正的变成负的了没有？”

好，先停在这儿。我要特别强调这个顺序——**先靠直觉猜一猜，再看数学定义**。刚才你们猜的这几条，我都只回了一句“对”或者“再想想”，背后的道理一条都没展开——我们带着这几个悬念去看下一页。数学定义不是凭空来的，它就是把你们刚才眼睛里看到的东西，用三个数字写下来而已。

【幻灯片 24】**rvec 到底是什么** ⌚ 2.5 分钟

〔操作〕翻到第 24 页。

好，现在揭晓。刚才那三个数叫 `rvec`，它的正式名字叫**旋转向量**，英文 `rotation vector`。一句话，你们记一下：

它的方向，就是旋转轴的方向；它的模长，也就是这个向量的大小，就是绕这根轴转过的角度，单位是弧度。

记住，是**弧度**，不是度。弧度跟度的换算大家不陌生，180 度等于 π 弧度。

好，现在回过头，兑现刚才那两个问题。

第一个，为什么只绕一个轴转，只有一个分量在变？现在明白了吧——因为**旋转向量是有方向的**，而这个方向就是你的旋转轴。你绕竖直的那根轴转，这个向量的方向就正好沿着竖直轴，它在另外两个方向上的分量自然就是零，所以只有对应那一个方向的分量在动。你刚才观察到“只有一个数在变”，这不是巧合，**这就是旋转向量的定义本身**。

〔停顿〕给学生两秒钟回味。

第二个，为什么转得越多数值越大、还会变号？因为**模长就是角度**。你转的角度越大，这个向量的长度就越长，那个在变的分量也就跟着涨——这正好解释了刚才你看到的现象：那个数随着手转过去一路变大。至于变号——还是绕这根竖直轴，你往左转 30 度和往右转 30 度，旋转轴的方向正好是反的，整个向量翻了个个儿，那个分量就从正变负了。

所以你看，`rvec` 这三个数不是随便蹦出来的，每一个数都在描述“往哪个方向转、转了多少”。

那问题来了——为什么 OpenCV 要用三个数来表示旋转，而不是用那个更常见的、 3×3 的旋转矩阵？

〔停顿〕等一下。

因为省。任意一个三维旋转，其实只有三个自由度，用三个数就够了。 3×3 的矩阵有九个数字，但里面是冗余的，它背着一堆约束条件。`rvec` 三个数，不多不少，刚刚好。

那既然三个数就够，为什么下一页还要出现矩阵、欧拉角、四元数这些别的说法？我们来看。

【幻灯片 25】旋转的几种表示法，一张图看懂 ⌚ 2.5 分钟

〔操作〕翻到第 25 页。

这一页就一张图，但信息量很大。我们把图上的三条箭头一个一个看清楚。

第一条，**旋转向量 `rvec`，经过 `cv::Rodrigues` 这个函数，得到旋转矩阵 `rmat`**。这个转换用的就是**罗德里格斯公式**。注意这条线是**可逆的**——你有一个 `rvec`，能算出 `rmat`；反过来，你有一个 `rmat`，也能用同一个函数算回 `rvec`。它俩一一对应，转过去、转回来，不丢信息。

〔板书〕黑板上写：`rvec -- (cv::Rodrigues) --> rmat`，画成双向箭头。

第二条，**旋转矩阵 `rmat`，经过反三角函数，得到欧拉角**，也就是我们熟悉的 `yaw`、`pitch`、`roll` 这三个角。这条线的意思是：你手上有一个 3×3 的矩阵，想把它翻译成“先绕哪个轴转多少、再绕哪个轴转多少、最后绕哪个轴转多少”，用反三角函数就能解出来。

第三条，**旋转矩阵 `rmat`，经过一组转换公式，得到四元数**。四元数就是四个数，`w`、`x`、`y`、`z`，这个我们下一页还会碰到。

所以你看，**rvec**、**旋转矩阵**、**欧拉角**、**四元数**，这四个东西本质上是等价的——它们说的都是同一件事，就是“这个物体到底是怎么转的”，只是换了一身衣服。什么时候用哪一套，看场景：**rvec** 省地方，欧拉角给人看最直观，四元数在连续旋转和插值的时候最稳。

那我们这一讲要走哪条路呢？是 **rvec** 先变成 **rmat**，再从 **rmat** 变出欧拉角这一条。为什么？因为等一下 P5 里的 Task 05，要的就是这个——它让你用 `cv::Rodrigues` 把 **rvec** 转成旋转矩阵，再用反三角函数把 yaw、pitch、roll 解出来。所以下一页，我们就把最后这一步讲透。

【幻灯片 26】从旋转矩阵提取欧拉角（INT_YXZ 约定） ⌚ 3.5 分钟

〔操作〕翻到第 26 页，标题是“从旋转矩阵提取欧拉角（INT_YXZ 约定）”。这一页是本讲最重要的理论页，速度放慢。

先看标题里那个缩写，**INT_YXZ**。这是这一页所有公式的前提，叫**旋转顺序**，也叫**约定**。

为什么非得先约定？因为欧拉角不是一步转到位，它是**分三步转**的：先绕一根轴转一个角，再绕第二根轴转一个角，最后绕第三根轴转一个角。Y、X、Z 就是这三根轴的先后顺序——先绕 Y、再绕 X、最后绕 Z。**顺序一换，同一个姿态算出来的三个数就完全不一样**。所以公式表头才要标 INT_YXZ。记住：**用公式之前，先认约定**。

好，约定认完了，我们看公式。先说一个记号：我们把旋转矩阵 **rmat** 的第 *i* 行、第 *j* 列这个元素，记作 **m** 下标 **ij**。整张表就三个式子：

```
// rmat 的第 i 行第 j 列元素记作 m_ij
yaw   = atan2( m13, m33 )
pitch = -asin( m23 )
roll  = atan2( m21, m22 )

// 注意：atan2 / asin 算出来是弧度，乘 57.3 (≈180/π) 换算成角度
```

我们一个一个看。

第一个，为什么用 **atan2**？**atan2** 叫**两参数的反正切**，是 C++ 标准库里就有的函数——`std::atan2(double, double)`，吃两个参数：分子的 y、分母的 x。它比旁边 **asin** 的值域宽得多——**atan2** 能给出负 180 度到正 180 度，一整圈都能表示；**asin** 只能给出负 90 到正 90 度，半圈。原课专门强调过，**atan2** 是两参数的反正切，这个一定要记住。

第二个，pitch 用的是 **asin**，反正弦。那它前面为什么带个负号？这也是**约定**带来的——因为我们这套公式是按 YXZ 的顺序推出来的，推到最后符号就落成了这个样子。你换一套约定，这个负号可能就没了。所以你不用去背这个负号的来路，你只要知道：**它是约定的一部分，跟着约定走**。

第三个，roll，也是 **atan2**，取的是 **m21** 和 **m22**。

〔停顿〕让学生把三个式子抄下来。

好，公式抄完了。接下来这一条，是我今天最想让你们记住的一步，也是**往届学生最容易错的地方**。

你们注意看公式里这些下标：**m13**、**m33**、**m23**、**m21**、**m22**。这些下标**都是从 1 开始数的**——**m13**，就是第 1 行、第 3 列。

但是，我们代码里的 `rmat` 是一个 `cv::Mat`。`cv::Mat` 的下标，是从 0 开始的。后面 Task 05 的代码注释里专门写了这一句：**`cv::Mat` 的下标从 0 开始，而不是 1。**

〔板书〕左边一列写公式下标，右边一列写代码下标，中间画对应线。

所以，公式里的 `m13`，在代码里要写成 `rmat.at<double>(0, 2)`——你看，第 1 行变成 0、第 3 列变成 2，**每个下标都减了一**。同理，我们把这五个都对应上：

〔板书〕 `m13` → `rmat.at(0, 2)` `m23` → `rmat.at(1, 2)` `m21` → `rmat.at(1, 0)` `m33` → `rmat.at(2, 2)` `m22` → `rmat.at(1, 1)`

就差这一个 1，数错一个，全盘皆错。你们等一下在 P5 写 Task 05 的时候，如果算出来的角度莫名其妙、跟实际转的方向对不上，第一件事就回来查这五个下标，看是不是忘了减一。这是我们今天反复要强调的一个点。

最后看一下幻灯片最后那行注释。它写着：`atan2`、`asin` 算出来是**弧度**，乘 57.3——也就是 180 除以 π ——换算成角度。

这里我要提醒一句口径：**我们工程里的代码，是把弧度直接显示出来的，没有乘 57.3。**所以等一下你在画面上看到的 `yaw`、`pitch`、`roll`，都是很小的数，零点几这种，那就是弧度。幻灯片上这句“乘 57.3”，是给你们的方法——**你想看度数，就自己乘 57.3。**千万别把这两个口径搞混了，别以为屏幕上打出来的是度数。

【幻灯片 27】欧拉角的坑：万向锁 ⌚ 2 分钟

〔操作〕翻到第 27 页。左右两栏，一栏欧拉角，一栏四元数，对比着看。

好，P4 最后一页，讲欧拉角的坑。

左边这一栏，欧拉角的好处是**直观、好理解**——“飞机抬头 30 度”，谁都听得懂。但它有两个短板。第一，**表示不唯一**——换个旋转顺序，同一个姿态算出来的三个角就完全不同，所以别人给你一组欧拉角，你不问约定就没法用。第二，就是这页标题——**万向锁，Gimbal Lock**。名字听着吓人，道理很直观：欧拉角就像**三个环套在一起**，正常时各管一个方向的转动。可当你转到某个特定角度，比如中间那个环转到了 90 度，**两个环就重合到一起了**——本来它俩管两个方向，现在只管一个方向了。**三个自由度，一下子丢了一个。**

〔停顿〕留两秒。

丢一个自由度意味着什么？意味着在这个姿态附近，你往那个丢了的方向怎么转，角度都表示不出来，运动就“卡住”了。这就是万向锁，点到为止，今天不推导。

右边这一栏的四元数，就是来解这个问题的：四个数 `w`、`x`、`y`、`z`，**没有奇异点**，连续旋转、插值特别稳；缺点是不直观。今天也不展开。

不过这个挺好玩的，我现场带你们玩一下。

〔演示〕打开浏览器，访问 quaternions.online，投到屏幕上。

看屏幕。左边一列是 Quaternion，四个输入框 `W`、`X`、`Y`、`Z`；右边一列是 Euler Angles，三个输入框 `X`、`Y`、`Z`，旁边一个下拉框叫 XYZ-Order，就是刚说的**旋转顺序**；中间是三维网格，红 `x`、绿 `y`、蓝 `z`。我先动右边欧拉角，把 `X` 调大——网格跟着转了；再调 `Y`、再调 `Z`，都实时跟着动。现在我把左边四元数的 `W`、`X`、`Y`、`Z` 也调一遍，让姿态跟刚才**一模一样**——看到没有？**两组完全不同的数，描述同一个姿态。**这就是“本质等价”。

〔停顿〕网络不好就接一句：“没关系，地址我放群里——quaternions.online，课后自己去玩十分钟。”

这个是课后自己玩，**不是任务**，不用交。

好，P4 就到这儿，一句话收尾：**rvec 方向是旋转轴、长度是转角；经 Rodrigues 变成矩阵，再经反三角函数变出欧拉角；欧拉角有万向锁，四元数能避开它。**

下面进入 P5，动手实验二——把 yaw、pitch、roll 真正打到画面上。

【幻灯片 28】P5 动手实验二 · Task 04~05 ⌚ 0.5 分钟

〔操作〕翻到第 28 页章节页，标题打到投影上。

好，上一段 P4，我们把 rvec 这个东西彻底拆开了——方向是旋转轴、模长是转角，还讲透了它和旋转矩阵、欧拉角、四元数之间的关系。那一段是“看懂”。

这一段 P5，还是老规矩，你们自己动手，我们把它变成“手上会做”。一共就两个 Task，Task 04 和 Task 05，全在同一个文件里——`lecture4/yolo/src/main.cpp`，别的文件一个字都不用碰。

一句话概括我们要干的事：**先把 solvePnP 解出来的 tvec、rvec 打到画面上，再把那个看不懂的 rvec，翻译成 yaw、pitch、roll 三个能读的角。**上一段我在 PPT 上讲的那条链路，这一段就要靠你们自己的手，一行一行敲出来。开始。

【幻灯片 29】Task 04 · 把 tvec / rvec 打印到画面上 ⌚ 4 分钟

〔操作〕翻到第 29 页，把代码块放大。

看第 29 页，Task 04。大家把代码往下拉，找到标着 Task 04 的那两行 `draw_text`。你现在看到的是这个：

```
tools::draw_text(img, fmt::format("tvec: x{: .2f} y{: .2f} z{: .2f}", 0.0, 0.0, 0.0), cv::Point(10, 60), cv::Scalar(0, 255, 255), 1.7, 3);
tools::draw_text(img, fmt::format("rvec: x{: .2f} y{: .2f} z{: .2f}", 0.0, 0.0, 0.0), cv::Point(10, 120), cv::Scalar(0, 255, 255), 1.7, 3);
```

〔停顿〕在参数列表上画一圈。

先看这个函数的参数顺序，这里有个坑，我要专门讲三十秒。

`draw_text` 的完整签名是 `(img, text, point, color, font_scale, thickness)`——颜色在前，字号在后。你们要是以前见过别的写法，顺序可能是反的，写反了编译器会报 `cannot convert 'cv::Scalar' to 'double'`。这个报错看着吓人，其实就是位置放错了。

但你不用管这几个参数。它们已经是写好的，你一个都不要动。

你的任务就一句话：**改写 `fmt::format` 里那三个 0.0，把真正解出来的 tvec 和 rvec 打到画面上。**现在它俩打印的全是 0.0——那是我写死的占位符，不是真值。上一段 Task 03 你辛辛苦苦解出来的数据，就在 `rvec`、`tvec` 这两个变量里躺着，得把它请出来。

〔停顿〕把这句话钉一下。

我再说一遍，因为每年都有人在这里多改东西：**你只改 `fmt::format` 括号里的三个 0.0，别的一个字符都不要碰。**格式串 `{: .2f}` 不用动，参数个数还是三个，`cv::Point` 那串照旧。

那怎么把一个数从 `tvec` 里取出来？关键就在幻灯片上这句提示：`tvec.at<double>(0)`。我拆开讲。

tvec 是 solvePnP 吐出来的一个 3×1 的 `cv::Mat`。注意，它是 **cv::Mat**，不是数组，也不是 **vector**。`cv::Mat` 是 OpenCV 里存矩阵的容器，你要从里面取一个元素，不能像数组那样中括号一括，得用 `.at<类型>(下标)` 这个函数。

- 尖括号里的 `<double>`，是告诉它：我要取的这个元素是个 double。tvec 内部存的就是 double。
- 后面圆括号里的 0，是下标，表示取第几个元素。
- 所以 `tvec.at<double>(0)` 得到的是一个 double，就是 tvec 的第一个元素。

三个下标 0、1、2，正好对应 x、y、z 三个方向——0 是 x，1 是 y，2 是 z。rvec 一模一样的写法：`rvec.at<double>(0)`、`(1)`、`(2)`，同样是 x、y、z。

所以你要做的，就是把原来的三个 0.0，换成 `tvec.at<double>(0)`、`tvec.at<double>(1)`、`tvec.at<double>(2)`；下面那行 rvec 照做一遍。

这里我必须钉死一件事，别想偏了：**Task 04 就是把数字打印到画面上，就这一件事，没有别的**。这一页的核心，是让你亲手体会一次——**怎么从 cv::Mat 里，把一个元素抠出来**。这个动作，紧接着的 Task 05 你还要再用一次，而且一抠就是好几个。

〔操作〕停在这里，让大家开始改代码。

〔演示〕老师也切到自己机器上，一边改一边念：把三个 0.0 换成 at 调用，保存，重新编译运行。

好，敲起来。给大家五分钟。前面几个错的点我提前标一下：下标别写成 1、2、3；别把 `.at<double>` 漏掉变成中括号；格式串里有三个 `{}`，后面就必须跟三个参数，个数要对上；还有，画面上是三行字，**前两行才是 tvec 和 rvec**，第三行是下一个 Task 的，别改错行。

改完的同学，先别急着往下看，自己盯着画面确认两件事：第一，这两行数是不是不再是整整齐齐的 0.00 了；第二，你拿手把装甲板往前推一推，tvec 那三个数是不是立刻跟着动。都对上了，这个 Task 就算过。

【幻灯片 30】Task 05 · rvec → rmat → 欧拉角 ⌚ 4.5 分钟

〔操作〕翻到第 30 页，把代码块放大。

好，第 30 页，Task 05。这是本讲最后一个 Task，也是最综合的一个。它把我们 P4 讲的整条链路——rvec 到 rmat 到欧拉角——真正落到代码上。分三步走。

第一步，**声明旋转矩阵，调用 Rodrigues**。

```
cv::Mat rmat;
cv::Rodrigues(rvec, rmat);
```

`cv::Mat rmat;` 是开一个矩阵变量，用来装我们的旋转矩阵。然后 `cv::Rodrigues(rvec, rmat);`——把旋转向量 rvec 转成旋转矩阵 rmat。P4 那张流程图上第一条箭头，就是这一行代码。记住，这个函数是帮你把 rmat 原地填好的，你不用管它内部怎么算。

第二步，**套 INT_YXZ 公式，算三个角**。公式上一段刚在 PPT 第 26 页讲过，我念一遍，你对着看：

```
yaw    = atan2( m13, m33 )
pitch  = -asin( m23 )
```

```
roll = atan2( m21, m22 )
```

关键在这儿——公式里的 **m13、m23** 这些，都是“第几行、第几列”，是从 1 开始数的；可 `cv::Mat` 的下标是从 0 开始数的。所以每一个下标都要减一。这个对应关系，上一段我带着你们推过，这一页的提示里也写了 `rmat.at<double>(0, 2)`。现在我不直接给你答案，你自己对着写：m13 是第 1 行第 3 列，到代码里是几？m23 呢？m21、m33、m22 呢？

〔停顿〕二十秒左右，等学生把五个对应关系动笔写完。

m13 是第 1 行第 3 列，代码里就是提示里给的那个例子 `rmat.at<double>(0, 2)`——行、列都减了一。剩下的 m33、m23、m21、m22，你自己按同一个规矩写，写完对着提示里的例子检查，先别问、也别看旁边同学的。

写的时候盯住这个格式：`rmat.at<double>(行, 列)`，两个下标，中间一个逗号——这就是 `cv::Mat` 取二维元素的方式。它比刚才 `tvec` 多一个下标，因为矩阵是二维的，得分行、列。五个下标你全对上了，公式这一关就算过了。

第三步，把这三个变量，喂给第三行 `draw_text`。还是老动作——把 yaw、pitch、roll 这三个变量，填进第三行 `draw_text` 的参数里，格式串不用动。

画面上第三行，就是这三个角。

好，这里有一个口径我要钉死，特别容易讲混：画面上显示的这三个角，是弧度，不是度数。因为 `atan2`、`asin` 算出来天然就是弧度。所以你会看到 yaw 大概是 0.1 这种很小的数——小，不代表错，它就是弧度。想看度数怎么办？上一段 P4 第 26 页的幻灯片上写过——乘 57.3，也就是乘 180 除以 π ，就换成度了。这个换算你心里有数就行，代码这一版不带乘，我们先看弧度。

那我考考你们——

「yaw 现在大概是 0.1，有谁知道，这是多少度？」

〔停顿〕等学生心算。

〔应答〕·学生答“乘 57.3，大概 5.7 度”→“漂亮！0.1 乘 57.3 差不多就是 5.7 度。你看，弧度这个数看着小，换过来其实就是个很正常的偏角。”·学生答“5 度左右”→“方向对了。精确说 5.7 度左右，因为 1 弧度大约 57.3 度。”·学生答“不懂怎么算”→“就是乘 57.3。0.1 乘 57.3，五点几度，你按一下计算器就知道了。”·冷场 → 老师自己接：“我算一下——0.1 乘 57.3，大概 5.7 度。这个 57.3 就是 180 除以 π ，记一下。”

常见错误几个，我提前标出来：行列搞反——把 `(0, 2)` 写成 `(2, 0)`，这俩元素数值不一样，出来全是错的；忘了公式下标要减一，直接写 `(1, 3)`，可 `cv::Mat` 第三列是 2 不是 3；pitch 前面的负号丢了，`pitch = -asin(m23)`，那个负号不能少；还有就是以为显示的是度数，不是，是弧度。

好，动手。

〔操作〕停在这里，让学生写 Task 05，然后编译、运行。

【幻灯片 31】重投影误差：给点序配一把尺 🕒 3 分钟

〔操作〕翻到第 31 页。先别讲，等大家把 Task 05 跑起来。

好，都跑起来了吗？跑起来了，我们回来补一页——这一页是今天最实用的一页。

前面在 P3 第 18 页，我讲点序陷阱的时候说过一句话：**顺序写错，程序不报错、画面正常、数值还有值**。当时我说「有一种更硬的办法能判死它」。现在我来兑现。

〔停顿〕

这个办法叫重投影误差。名字听着唬人，思路特别朴素。

你想，我们已经解出了位姿——也就是知道了「这块装甲板在相机坐标系下的位置和朝向」。那反过来，我是不是可以拿**这个位姿，把那四个 3D 点重新投影回图像上，看看它们落在哪？**

如果位姿是对的，投回来的四个点，应该**正好落在检测器给出的那四个点上**——因为位姿就是从它们算出来的。

如果位姿是错的，投回来的点就会**偏到别的地方去**。

〔板书〕 `object_points` --(解出的位姿)--> 投影回图像 --对比--> `img_points`

那怎么衡量「偏了多少」？就是量距离——投回来的点，和检测器给的原始点，中间差几个像素。四个点取个平均，就是这个**重投影误差**。

〔停顿〕 让大家消化一下这个思路。

现在给你判读口径，记住这一句就够了：

个位数像素，就是对的；两位数，就是错的。中间没有灰色地带。

顺序写对的时候，这个数通常在 3 像素上下。你把 `object_points` 的第 2、4 点对调一下试试——它会直接跳到几十。

〔操作〕 切到教师机运行中的参考版，指一下画面上的 `reproj_err` 一行。

看我这台机器，画面上比你们**多一行**，写着 `reproj_err`。你们自己那份没有这一行。

为什么你们没有？因为我没把它放进学生版——它是自查工具，不是任务。

那你们想用怎么办？特别简单：在 `main.cpp` 顶上加一行 `#include "tools/pnp_check.hpp"`，然后在画完欧拉角之后调一下 `tools::reprojection_error(...)`，把返回值画到画面上。**这是个 header-only 的工具，不用改 CMake，加一行 include 就能用。**

〔停顿〕

我为什么专门花三分钟讲这个？

因为今天这一课，最值钱的不是 `solvePnP` 那六个参数——那个你查文档就会了。最值钱的是**「当你手里的工具不给你报错的时候，你怎么自己造一个判据」**。点序错了没人告诉你，但你可以自己造一把尺去量。

这页和 P3 那页点序陷阱是配对的——一个说坑在哪，一个给出路。只讲坑不给路，那叫吓唬人。

好，翻到第 32 页，我们收个尾。

【幻灯片 32】P5 · 现场演示 & 互动 ⌚ 4 分钟

好，我们做个小实验，答案马上就在屏幕上。

〔演示〕指着投影上运行中的程序窗口，画面左上角三行数字清晰可见。

我不动板子了——画面上那只手拿着的，就是我们的装甲板，程序在循环播那段视频。我们一起盯着看。

〔停顿〕让视频转一到两轮。

看到没有？画面上第三行，yaw、pitch、roll，跟着那只手的动作在实时变。板子往左扭，yaw 动；抬头低头，pitch 动；侧向歪一下，roll 动。**这就是你亲手算出来的三个欧拉角**。从像素，到 rvec，到 rmat，再到这三个数——整条链路，你通了。

〔停顿〕按空格停在一个姿态上，方便大家看清三行数。

来，请一两位同学上来，用空格键停住画面，一边对照一边把规律讲给大家听。

〔演示〕邀请 1~2 位同学上台操作空格暂停/继续，描述 yaw/pitch/roll 谁在变、怎么变；老师负责接话、纠正。

好，先停一下，我们看一个核心问题，都在 PPT 上，我念一下原话：

「把装甲板前后移动（不转动），tvec 和 rvec 分别会怎么变？」

先别急着答，你可以现在就试——我等你五秒。

〔停顿〕等五到八秒，看有没有人举手。

〔应答〕· 学生答"tvec 变、rvec 基本不变" → "完全正确。往前推，位置变了，tvec 就动；可你压根没转它，朝向没变，rvec 当然稳。"· 学生答"两个都变" → "再盯一眼屏幕——rvec 那三个数是不是基本定住了？会有一点点抖，那是噪声，但本质上它是没动的。"· 学生答"只有 tvec 变" → "对。那你说说，为什么 rvec 不变？"（引导他答出"因为没旋转"）· 冷场 → 老师自己接："我给答案——**tvec 变、rvec 基本不变**。位置动了，朝向没动，就这八个字。"

〔演示〕让视频里的手把装甲板往前推、往后拉的那几秒，按空格停住，手指着屏幕上的 tvec 和 rvec 两行作对照。

我们再反过来验证一次，这个特别有用。现在，**装甲板原地转——只转，不动位置**。你们看，rvec 变了，三个数跟着转；那 tvec 呢？基本稳住，纹丝不动。

〔停顿〕对照这两行数。

所以这张关系你们记牢：**平移，动的是 tvec；旋转，动的是 rvec**。位置和朝向，在数学上分工明确、各管一段——这就是我们一开始说的"位姿 = 位置 + 朝向"，现在你亲眼看到它俩是分开的。

【巡场话术库】Task 04~05 通用，老师走下讲台时用

〔操作〕在学生中间走动，遇到下面几类情况，照这个思路说。

情况一：画面上数值一直是 0.00。「你这是 Task 04 还没动过吧？看那两行 draw_text，后面是不是还挂着 0.0、0.0、0.0？把三个 0.0 换成 `tvec.at<double>(0)`、`(1)`、`(2)`，rvec 那行照做。改完保存、重新编译。」

情况二：编译报错，卡在 at 这个模板上。「报错里出现 at 了是吧？多半是尖括号或者圆括号写漏了。正确写法是 `tvec.at<double>(0)`——.at 后面尖括号里写 double，圆括号里写下标。别写成中括号 `at[double]`，

也别漏了那对尖括号。还有，取二维元素要两个下标，比如 `rmat.at<double>(0, 2)`，中间的逗号别丢。」

情况三：欧拉角数值不对，或者行列搞反了。「算出来某个角一直是 0，或者是 nan，或者跟板子的转动方向对不上？先查下标顺序。公式里是 m_{13} ，第 1 行第 3 列，代码里就是 `(0, 2)`——别写成 `(2, 0)`，那两个数不一样。再检查一下，是不是每个下标都忘了减一。这俩是重灾区。」

情况四：以为角度单位是度数。「你盯着那个 0.1 发愁，觉得太小了？它是弧度，不是度。弧度就长这样，0.1 大概是五点几度。想看度数，乘 57.3。代码这一版不乘，我们就看弧度。」

【集合收尾话术】过渡到 P6

〔操作〕拍两下手，把注意力收到投影上，切到第 33 页。

「好，大家先停一下手上的活，看我这儿。没跑完的保存一下，一会儿接着弄。」

这一段上机，我们完成了一件挺了不起的事：**把 solvePnP 解出来的位姿，从三个看不懂的 `rvec`，变成了 `yaw`、`pitch`、`roll` 三个能读的角**。再加上上一段的 `tvec`，画面上三行数字，位置有了、朝向也有了——你的第一块装甲板位姿，正式算出来了。

但是，你盯着这三行数，心里应该还悬着一个问题：**这三个角，是相对谁的？是相对相机坐标系的**。那再往下问一句——相机装在哪？它跟枪管是在一条线上吗？它跟机器人本体又是什么关系？

带着这个问题，我们进入最后一块拼图——坐标系全景。今天我们已经把像素到相机这一环打通了，接下来看看从这个相机坐标系再往外走，到机器人本体、再到 IMU，中间还差着几环。」

【幻灯片 33】P6 章节页 · 坐标系全景 ⌚ 0.5 分钟

〔操作〕翻到第 33 页，这是 P6 的章节页。

好，上机先告一段落，把目光收回到投影上。Task 01 到 Task 05 大家都跑通了：`solvePnP` 解出了 `tvec`、`rvec`，我们又用 `cv::Rodrigues` 和反三角函数，把装甲板的朝向变成了 `yaw`、`pitch`、`roll` 三个角。

可是我想先按一下暂停，问一句：我们算出来的这些数，到底是站在**哪个坐标系**里说的？它离“真的对准”，还差几步？

这就是 P6 要回答的问题——坐标系全景。

【幻灯片 34】我真的对准了吗？ ⌚ 2.5 分钟

〔操作〕翻到第 34 页，标题是“我真的对准了吗？”。

在回答刚才那个问题之前，我们先把一个概念钉死：什么叫“**相机坐标系**”。

大家回忆一下前面讲过的——相机坐标系，是跟相机**刚性连接**的一个坐标系。什么叫刚性连接？就是它跟相机是一体的：相机往左移，它跟着往左移；相机往上抬，它跟着往上抬；相机转个角度，它跟着一起转。相机和这个坐标系之间，不会发生任何相对运动。

那它的原点在哪？就在**镜头的光心**上。换句话说，今天画面上那个 `tvec`——装甲板中心相对相机的位置——它的参照点，就是镜头的光心。而在相机的坐标系里， x 向右、 y 向下、 z 指向镜头正前方。

好，现在我要抛一个思考题，这个题很关键，大家认真想一下：

「如果相机在装的时候是斜的，而且它和枪管不在同一个位置，会发生什么？」

〔停顿〕停三到五秒，扫一眼教室，等一两个同学开口。

〔应答〕·学生答"那 solvePnP 就白算了" → "先别急着怪算法。solvePnP 只负责把像素算成相机坐标系里的位姿，这一步它没做错；出问题的是——我们默认这个位姿就能直接拿去开火。"·学生答"会有偏差、打不准" → "对，方向没错。那我追问一句：这个偏差，是 solvePnP 算错了吗？还是别的什么东西我们压根没考虑？"·学生答"不知道"／冷场 → 老师自己接："我给个提示——今天所有的数，都是相对谁算出来的？"

答案其实就藏在我们今天的每一行输出里：我们算的，是装甲板在**相机坐标系**下的位姿。可真正要开火的是枪管，不是相机。相机是斜着装的，枪管是正的；相机装在车顶，枪管在下面——这两个东西根本不在一条线上。那你算出来的"装甲板正对着相机"，和"装甲板正对着枪管"，能是一回事吗？不是。

所以结论请大家记一下：**光有"装甲板在相机坐标系下的位姿"还不够，你还得知道相机相对枪管、相对机器人本体的安装关系。**这句话，就是从"我算对了"到"我打中了"之间，必须补上的那一段。

【幻灯片 35】完整的坐标变换链条 ⌚ 3 分钟

〔操作〕翻到第 35 页，这是今天格局最大的一页。

好，那我们就干脆把整条链路，像一张图一样，铺在大家面前。

先从最原始的那一层说起——**像素坐标系**。就是我们图像上那一堆点：左上角是原点，横着是 u ，竖着是 v ，单位是像素。识别器吐给我们的那四个点，就活在这一层。它最直观，但也最"虚"——没有物理尺度，也没有方向。

往上走一步，是**相机坐标系**。这一步靠什么跨过去？就是我们今天学了一整天的 **solvePnP**。四个三维点、四个二维点，加上相机内参和畸变系数喂进去，它就还给我们一个 $tvec$ 和一个 $rvec$ ，告诉我们装甲板在相机坐标系里在哪、朝哪。**这一环，今天打通了。**

再往上，是**机器人本体坐标系**——固定在这台车自己身上的一个坐标系，比如车体中心。为什么还要往上走？因为相机是装在车上的，相机眼里的世界，得翻译成"车自己觉得世界是什么样"，才能拿去控制云台、控制枪管。那这一层靠什么跨过去？**手眼标定**。手是云台、是枪管，眼是相机，标的就是相机相对机器人本体那组变换。

〔板书〕黑板或白板上画四个方框：像素 → 相机 → 本体 → IMU，方框之间分别标上 solvePnP、手眼标定、实际测量安装。

手眼标定这东西，原课给大家看过一张图：相机装在机械臂的末端，机械臂动一动，相机跟着动，通过解一组 $AX = B$ 的方程，把相机和本体之间的相对变换解出来。这是个进阶内容，今天不展开，但你们要知道有这么一步，也要知道它为什么难——因为相机和手之间，隔着一整个机械结构的安装关系。

再往上，最后一层，是**IMU 坐标系**。IMU 是车上的惯性测量单元，测的是姿态、加速度、角速度。为什么要跟它对齐？因为机器人本体的控制是依赖 IMU 陀螺仪的，你需要知道 IMU 到本体坐标系之间的那个变换。而这一层，靠的是**实际测量安装**——IMU 装在车上的哪个位置、朝哪个方向，是量出来的，不是算出来的。

所以整条链，一句话串起来：**像素坐标系 → 相机坐标系 → 机器人本体坐标系 → IMU 坐标系**，分别靠 **solvePnP**、**手眼标定**、**实际测量安装** 这三步跨过去。

那这句话要划重点：**今天，我们只打通了第一环——从像素到相机。**手眼标定，还有后面基于卡尔曼滤波的运动预测，都留给后续课程。所以你们今天算出来的位姿，它的参照原点，站在**相机**上。这句话请记住：今天的结果，只到相机坐标系。

好，坐标系全景就到这儿。我们把目光收回来，做今天的总结，把作业布置下去。

【幻灯片 36】P7 章节页 · 总结与作业 ⌚ 0.25 分钟

〔操作〕翻到第 36 页，语速放慢，给学生一个“要收尾了”的信号。

好，我们来到今天最后一段——P7，总结与作业。这一段只做三件事：先一起把今天的链路对一遍答案，再把作业布置清楚，剩下的时间，全部留给大家提问。

〔操作〕稍作停顿，翻到第 37 页。

【幻灯片 37】P7 · 本讲回顾 ⌚ 1.75 分钟

好，别急着关电脑，我把这节课对一遍答案。八条，我起个头，你们在心里接。

第一条，位姿等于位置加朝向——位置管“在哪”，朝向管“对着哪”，两个都在，才叫位姿。

第二条，PnP 就是：n 组——对应的 3D 点和 2D 点，加上相机内参，解出旋转 R 和平移 t。

第三条，`cv::solvePnP` 输入四样数据——物体点、图像点、相机内参、畸变系数，输出两样——`rvec` 和 `tvec`。

第四条，`tvec` 就是装甲板坐标系的原点、也就是板中心，在相机坐标系下的位置。

第五条，这条是今天新学的重点，务必记住——`rvec` 过一遍 Rodrigues 就得到旋转矩阵 `rmat`，`rmat` 再过反三角函数就得到欧拉角，换一组公式还能转成四元数；这几个东西本质上是等价的，说的都是同一件事，这块板子是怎么转的。

第六条，同样是今天的新重点——欧拉角有它自己的坑，就是万向锁：到了特定姿态，它会丢掉一个自由度。四元数没有这个问题，做连续旋转、做插值都更稳健。

第七条，`object_points` 和 `img_points` 必须一一对应。写错了，程序不报错、画面正常、数值还有值——但位姿是全错的。

第八条，所以要会自己造判据：**重投影误差**。把算出来的位姿投影回图像量像素差，个位数是对的，两位数是错的。今天真正想教给你的不是背下这个工具，是那种意识——**工具不报错的时候，自己造一把尺去量**。

第九条，完整链条是：像素到相机靠 `solvePnP`，相机到机器人本体靠手眼标定，机器人本体到 IMU 靠实际测量安装——而今天，我们只打通了第一环。

顺便把这一路走过来的课也顺一遍——最早是 Ubuntu 命令行和 g++ 编译，接着 CMake，再到 OpenCV，然后是面向对象，上一讲是 Modern C++，今天轮到 PnP。一环一环搭上来，才走到今天的位姿解算。

好，八条对完了。第五条、第六条是今天新学、也最容易考，第七条、第八条是我最希望你们带走的，回去都再过一遍。有没有哪一条，你想让我再说一遍的？

〔停顿〕停两三秒，扫一圈教室。

〔应答〕·有学生说“第五条没跟上”→“好，那我再再说一次：`rvec` 先过 Rodrigues 变成 `rmat`，`rmat` 再过反三角函数就是欧拉角，换个公式就是四元数，它们是一回事。”·有学生说“第八条那个尺怎么用？”→“加一行 `#include <tools/pnp_check.hpp>`”，调 `tools::reprojection_error(...)`，把返回值画到画面上。header-only，不用改 CMake。”·冷场→“没有？那我再点一条——第六条，万向锁，还有它为什么让四元数更稳。”

〔操作〕翻到第 38 页。

【幻灯片 38】P7 · 课后作业 ⌚ 1 分钟

好，作业，三档，我按页面念，你们照着记。

必做，是底线：把课上没做完的 Task 01 到 Task 05 全部补完，跑通，让画面上正确显示出 tvec、rvec，还有 yaw、pitch、roll——注意，五个量都要出来，欧拉角那三个也是必做，不是选做。

验收标准我说清楚，别自己糊弄自己：**tvec 的 z 在 0.2 到 0.7 之间，跟着画面里装甲板的远近连续变，不乱跳**。不要拿「框画出来了」当判据——那行画框的代码在 Task 02 前面，你一个字不填它也画。

进阶，二选一：

选 A 的话，接真机——需要 HikRobot 相机和 MVS SDK，`-DWITH_HIKROBOT=ON` 重新配置，把 `configs/yolo.yaml` 的 `source` 改成 `camera`。**但有个坑**：工程里那组相机内参是给视频那台相机标的，你换了相机，得用自己的标定结果替换 `main.cpp` 顶部那两组数，不然重投影误差会暴涨。

选 B 的话，零硬件，人人可做——工程里 `lecture4/homework/` 那个目录，是一个**能量机关**的位姿解算脚手架，里面 `Buff_Solver::solvePnP()` 是个**空函数体**，就等你填。注意它 `tasks/CMakeLists.txt` 里的 OpenVINO 路径跟 yolo 工程写的**不一样**，改成你机器上的实际路径再编。

思考题，这一条我下节课要抽查——什么姿态下欧拉角会出现万向锁？你先自己猜一个，再上 quaternions.online 亲手验证，看你猜得对不对。下节课我点名让人上来讲。

还有一个可选的练习，我强烈建议做：故意把 `object_points` 的第 2、4 点对调，跑一遍，记下 `reproj_err` 从多少变成了多少——然后解释一下，为什么这种情况下 `tvec` 看起来**还是像一个正常数字**。这个练习没有标准答案，但它能检验你是不是真的懂了今天第七条和第八条。

一句话记牢：必做是底线，进阶是加餐，思考题我会查。

〔停顿〕留几秒让学生记。

【幻灯片 39】Thanks · Q&A ⌚ 弹性（0 分钟，不计入 3 分钟）

〔操作〕翻到第 39 页，切回轻松的语气。

好，今天的正课就到这儿，谢谢大家，剩下的时间交给提问。

顺便预告下一讲：**Hello Kalman && Target**——目标还在动，你得预测它下一刻在哪。

〔停顿〕

至于**手眼标定**，它是自瞄绕不过去的一步，但我们不会单独拿一讲来讲它——它会出现在后面某一段里。

现在开放提问，代码、概念、报错，都可以问。

〔停顿〕留白，扫一圈教室，等学生举手。

〔应答〕·有学生问“yaw、pitch、roll 是不是就是装甲板的朝向？”→“抓得很准。Task 05 算出来的这三个角，就是从旋转矩阵里提取、描述装甲板自身朝向的欧拉角，它是我们今天的核心，和 tvec 那个位置量分开放。”·有学生问“画面上这三个角数值怎么这么小？”→“因为它是弧度。PPT 上那句乘 57.3，就是告诉你想看度数就这么换算；我代码里没乘，所以显示的是弧度，这不是 bug。”·有学生问“多转几

次，欧拉角会不会突然跳？" → "会，这就是万向锁那一类的奇异问题，也正因如此我们才说四元数更稳健。你回去用 `quaternions.online` 一试就体会到了。" · 有学生问"我用了 `armor.left.top`，程序一跑就崩" → "症状对上了——你取错地方了。YOLO 这条路从不给 `left / right` 赋值，你拿到的是四个 (0,0)。改用 `armor.points.at(0)` 到 `.at(3)`。"

〔应答〕冷场时，老师主动抛砖引玉："没人问？那我先抛一个问题：**如果相机和枪管不在同一个位置、方向也不一致，光凭今天的 `tvec`，能不能直接开火？**换句话说，要让子弹真的打中，我们下一步还缺哪一层变换？"〔停顿〕等一等，没人接就自己接——"给个提示：今天的 `tvec` 说的是『装甲板相对相机』。可枪口在相机的位置吗？方向又跟相机一致吗？所以中间还差一层变换，这就是手眼标定。再往后，目标还在动，你还得预测它下一刻在哪——那就是下一讲的卡尔曼滤波。今天先想到这儿就够了。"

〔操作〕趁大家记笔记，顺势补一句："我把三个课后链接发到群里，想深入的自己拿。一个讲万向锁的知乎文章，一个补旋转矩阵背景的 B 站线性代数视频，还有一个是 OpenCV 里 `cv::Quat` 四元数类的官方文档。都不是作业，感兴趣再看。"

结束语

好，那今天的课我们就上到这里。整堂课其实就干了一件事——从图像上四个 2D 像素点出发，一路推出了装甲板在相机坐标系下的**位置和朝向**。记住那句话：**2D 检测解决"看见"，3D 位姿解决"打中"**。

回去把 Task 01 到 05 亲手调通，卡住的地方不要怕，错误是最好的老师；思考题别忘了做，下节课我第一个点你。

下课，同学们再见！